

Pointer

指针

变量的地址、指针

在程序中，我们的数据都有其存储的地址。在程序每次的实际运行过程中，变量在物理内存中的存储位置不尽相同。不过，我们仍能够在编程时，通过一定的语句，来取得数据在内存中的地址。

地址也是数据。存放地址所用的变量类型有一个特殊的名字，叫做「指针变量」，有时也简称做「指针」。

▼ 指针变量的大小

指针变量的大小在不同环境下有差异。在 32 位机上，地址用 32 位二进制整数表示，因此一个指针的大小为 4 字节。而 64 位机上，地址用 64 位二进制整数表示，因此一个指针的大小就变成了 8 字节。

地址只是一个刻度一般的数据，为了针对不同类型的数据，「指针变量」也有不同的类型，比如，可以有 `int` 类型的指针变量，其中存储的地址（即指针变量存储的数值）对应一块大小为 32 位的空间的起始地址；有 `char` 类型的指针变量，其中存储的地址对应一块 8 位的空间的起始地址。

事实上，用户也可以声明指向指针变量的指针变量。

假如用户自定义了一个结构体：

```
1 struct ThreeInt {
2     int a;
3     int b;
4     int c;
5 };
```

则 `ThreeInt` 类型的指针变量，对应着一块 $3 \times 32 = 96$ bit 的空间。

指针的声明与使用

C/C++ 中，指针变量的类型为类型名后加上一个星号 `*`。比如，`int` 类型的指针变量的类型名即为 `int*`。

我们可以使用 `&` 符号取得一个变量的地址。

要想访问指针变量地址所对应的空间（又称指针所 **指向** 的空间），需要对指针变量进行 **解引用**（`dereference`），使用 `*` 符号。

```
1 int main() {
2     int a = 123; // a: 123
3     int* pa = &a;
4     *pa = 321; // a: 321
5 }
```

对结构体变量也是类似。如果要访问指针指向的结构中的成员，需要先对指针进行解引用，再使用 `.` 成员关系运算符。不过，更推荐使用「箭头」运算符 `->` 这一更简便的写法。

```

1 struct ThreeInt {
2     int a;
3     int b;
4     int c;
5 };
6
7 int main() {
8     ThreeInt x{1, 2, 3}, y{6, 7, 8};
9     ThreeInt* px = &x;
10    (*px) = y;    // x: {6,7,8}
11    (*px).a = 4;  // x: {4,7,8}
12    px->b = 5;    // x: {4,5,8}
13 }

```

指针的偏移

指针变量也可以和整数进行加减操作。对于 `int` 型指针，每加 1（递增 1），其指向的地址偏移 32 位（即 4 个字节）；若加 2，则指向的地址偏移 $2 \times 32 = 64$ 位。同理，对于 `char` 型指针，每次递增，其指向的地址偏移 8 位（即 1 个字节）。

使用指针偏移访问数组

我们前面说过，数组是一块连续的存储空间。而在 C/C++ 中，直接使用数组名，得到的是数组的起始地址。

```

1 int main() {
2     int a[3] = {1, 2, 3};
3     int* p = a;    // p 指向 a[0]
4     *p = 4;        // a: [4, 2, 3]
5     p = p + 1;     // p 指向 a[1]
6     *p = 5;        // a: [4, 5, 3]
7     p++;           // p 指向 a[2]
8     *p = 6;        // a: [4, 5, 6]
9 }

```

当通过指针访问数组中的元素时，往往需要用到「指针的偏移」，换句话说，即通过一个基地址（数组起始的地址）加上偏移量来访问。

我们常用 `[]` 运算符来访问数组中某一指定偏移量处的元素。比如 `a[3]` 或者 `p[4]`。这种写法和对指针进行运算后再引用是等价的，即 `p[4]` 和 `*(p + 4)` 是等价的两种写法。

空指针

在 C++11 之前，C++ 和 C 一样使用 `NULL` 宏表示空指针常量，C++ 中 `NULL` 的实现一般如下：

```

1 // C++11 前
2 #define NULL 0

```

▼ C 语言对 `NULL` 的定义

C 语言在 C23 前有两个 `NULL` 的定义，只有类型不同：一个是整型常量表达式，一个是转换为 `void *` 类型的常量表达式，但其值都为 0，编译器可任选一个实现。

空指针和整数 0 的混用在 C++ 中会导致许多问题，比如：

```

1 int f(int x);
2 int f(int* p);

```

在调用 `f(NULL)` 时，实际调用的函数的类型是 `int(int)` 而不是 `int(int *)`。

▼ NULL 在 C 语言中造成的问题

比起在 C++ 中，因为有两个定义，在 C 语言中 `NULL` 造成的问题更为严重：如果在一个传递可变参数的函数中，函数编写者想要接受一个指针，但是函数调用者传递了一个定义为整型的 `NULL`，则会造成未定义行为，因在函数内使用传入的可变参数时，要进行类型转换，而从整型到指针类型的转换是未定义行为。¹

为了解决这些问题，C++11 引入了 `nullptr` 关键字作为空指针常量。

C++ 规定 `nullptr` 可以隐式转换为任何指针类型，这种转换结果是该类型的空指针值。

`nullptr` 的类型为 `std::nullptr_t`，称作空指针类型，可能的实现如下：

```
1 namespace std {
2 typedef decltype(nullptr) nullptr_t;
3 }
```

另外，C++11 起 `NULL` 宏的实现也被修改为了：

```
1 // C++11 起
2 #define NULL nullptr
```

▼ C 语言对空指针常量的改进

基于类似的原因，C23 也引入了 `nullptr` 作为空指针常量，同时引入了 `nullptr_t` 作为其类型¹。

指针的进阶使用

使用指针，使得程序编写者可以操作程序运行时中各处的数据，而不必局限于作用域。

指针类型参数的使用

在 C/C++ 中，调用函数（过程）时使用的参数，均以拷贝的形式传入子过程中（引用除外，会在后续介绍）。默认情况下，函数仅能通过返回值，将结果返回到调用处。但是，如果某个函数希望修改其外部的数据，或者某个结构体/类的数据量较为庞大、不宜进行拷贝，这时，则可以通过向其传入外部数据的地址，便得以在其中访问甚至修改外部数据。

下面的 `my_swap` 方法，通过接收两个 `int` 型的指针，在函数中使用中间变量，完成对两个 `int` 型变量值的交换。

```
1 void my_swap(int *a, int *b) {
2     int t;
3     t = *a;
4     *a = *b;
5     *b = t;
6 }
7
8 int main() {
9     int a = 6, b = 10;
10    my_swap(&a, &b);
11    // 调用后, main 函数中 a 变量的值变为 10, b 变量的值变为 6
12 }
```

C++ 中引入了引用的概念，相对于指针来说，更易用，也更安全。详情可以参见 [C++: 引用](#) 以及 [C 与 C++ 的区别：指针与引用](#)。

动态实例化

除此之外，程序编写时往往会涉及到动态内存分配，即，程序会在运行时，向操作系统动态地申请或归还存放数据所需的内存。当程序通过调用操作系统接口申请内存时，操作系统将返回程序所申请空间的地址。要使用这块空间，我们需要将这块空间的地址存储在指针变量中。

在 C++ 中，我们使用 `new` 运算符来获取一块内存，使用 `delete` 运算符释放某指针所指向的空间。

```
1 int* p = new int(1234);
2 /* ... */
3 delete p;
```

上面的语句使用 `new` 运算符向操作系统申请了一块 `int` 大小的空间，将其中的值初始化为 1234，并声明了一个 `int` 型的指针 `p` 指向这块空间。

同理，也可以使用 `new` 开辟新的对象：

```
1 class A {
2     int a;
3
4     public:
5     A(int a_) : a(a_) {}
6 };
7
8 int main() {
9     A* p = new A(1234);
10    /* ... */
11    delete p;
12 }
```

如上，「`new` 表达式」将尝试开辟一块对应大小的空间，并尝试在这块空间上构造这一对象，并返回这一空间的地址。

```
1 struct ThreeInt {
2     int a;
3     int b;
4     int c;
5 };
6
7 int main() {
8     ThreeInt* p = new ThreeInt{1, 2, 3};
9     /* ... */
10    delete p;
11 }
```

▼ 列表初始化

`{}` 运算符可以用来初始化没有构造函数的结构。除此之外，使用 `{}` 运算符可以使得变量的初始化形式变得统一。详见「[list initialization \(since C++11\)](#)」。

需要注意，当使用 `new` 申请的内存不再使用时，需要使用 `delete` 释放这块空间。不能对一块内存释放两次或以上。而对空指针 `nullptr` 使用 `delete` 操作是合法的。

动态创建数组

也可以使用 `new[]` 运算符创建数组，这时 `new[]` 运算符会返回数组的首地址，也就是数组第一个元素的地址，我们可以用对应类型的指针存储这个地址。释放时，则需要使用 `delete[]` 运算符。

```
1 size_t element_cnt = 5;
2 int *p = new int[element_cnt];
3 delete[] p;
```

数组中元素的存储是连续的，即 $p + 1$ 指向的是 p 的后继元素。

二维数组

在存放矩阵形式的数据时，可能会用到「二维数组」这样的数据类型。从语义上来讲，二维数组是一个数组的数组。而计算机内存可以视作一个很长的一维数组。要在计算机内存中存放一个二维数组，便有「连续」与否的说法。

所谓「连续」，即二维数组的任意一行（row）的末尾与下一行的起始，在物理地址上是毗邻的，换言之，整个二维数组可以视作一个一维数组；反之，则二者在物理上不一定相邻。

对于「连续」的二维数组，可以仅使用一个循环，借由一个不断递增的指针即可遍历数组中的所有数据。而对于非连续的二维数组，由于每一行不连续，则需要先取得某一行首的地址，再访问这一行中的元素。

▼ 二维数组的存储方式

这种按照「行（row）」存储数据的方式，称为行优先存储；相对的，也可以按照列（column）存储数据。由于计算机内存访问的特性，一般来说，访问连续的数据会得到更高的效率。因此，需要按照数据可能的使用方式，选择「行优先」或「列优先」的存储方式。

动态创建二维数组

在 C/C++ 中，我们可以使用类似下面这样的语句声明一个 N 行（row） M 列（column）的二维数组，其空间在物理上是连续的。

▼ 描述数组的维度

更通用的方式是使用第 n 维（dimension）的说法。对于「行优先」的存储形式，数组的第一维长度为 N ，第二维长度为 M 。

```
1 int a[N][M];
```

这种声明方式要求 N 和 M 为在编译期即可确定的常量表达式。

在 C/C++ 中，数组的第一个元素下标为 0，因此 $a[r][c]$ 这样的式子代表二维数组 a 中第 $r + 1$ 行的第 $c + 1$ 个元素，我们也称这个元素的下标为 (r, c) 。

不过，实际使用中，（二维）数组的大小可能不是固定的，需要动态内存分配。

常见的方式是声明一个长度为 $N \times M$ 的一维数组，并通过下标 $r * M + c$ 访问二维数组中下标为 (r, c) 的元素。

```
1 int* a = new int[N * M];
```

这种方法可以保证二维数组是 **连续的**。

▼ 数组在物理层面上的线性存储

实际上，数据在内存中都可以视作线性存放的，因此在一定的规则下，通过动态开辟一维数组的空间，即可在其上存储 n 维的数组。

此外，亦可以根据「数组的数组」这一概念来进行内存的获取与使用。对于一个存放的若干数组的数组，实际上为一个存放的若干数组的首地址的数组，也就是一个存放若干指针变量的数组。

我们需要一个变量来存放这个「数组的数组」的首地址——也就是一个指针的地址。这个变量便是一个「指向指针的指针」，有时也称作「二重指针」，如：

```
1 int** a = new int*[5];
```

接着，我们需要为每一个数组申请空间：

```
1 for (int i = 0; i < 5; i++) {  
2   a[i] = new int[5];  
3 }
```

至此，我们便完成了内存的获取。而对于这样获得的内存的释放，则需要进行一个逆向的操作：即先释放每一个数组，再释放存储这些数组首地址的数组，如：

```
1 for (int i = 0; i < 5; i++) {  
2   delete[] a[i];  
3 }  
4 delete[] a;
```

需要注意，这样获得的二维数组，不能保证其空间是连续的。

还有一种方式，需要使用到「指向数组的指针」。

▼ 数组名和数组首元素地址的区别

我们之前说到，在 C/C++ 中，直接使用数组名，值等于数组首元素的地址。但是数组名表示的这一变量的类型实际上是整个数组，而非单个元素。

```
1 int main() { int a[5] = {1, 2, 3, 4, 5}; }
```

从概念上说，代码中标识符 `a` 的类型是 `int[5]`；从实际上来说，`a + 1` 所指向的地址相较于 `a` 指向的地址的偏移量为 5 个 `int` 型变量的长度。

```
1 int main() {  
2   int(*a)[5] = new int[5][5];  
3   int* p = a[2];  
4   a[2][1] = 1;  
5   delete[] a;  
6 }
```

这种方式获得到的也是连续的内存，但是可以直接使用 `a[n]` 的形式获得到数组的第 `n + 1` 行 (row) 的首地址，因此，使用 `a[r][c]` 的形式即可访问到下标为 (`r`, `c`) 的元素。

由于指向数组的指针也是一种确定的数据类型，因此除数组的第一维外，其他维度的长度均须为一个能在编译器确定的常量。不然，编译器将无法翻译如 `a[n]` 这样的表达式 (`a` 为指向数组的指针)。

指向函数的指针

关于函数的介绍请参见 [C++ 函数](#) 章节。

简单地说，要调用一个函数，需要知晓该函数的参数类型、个数以及返回值类型，这些也统一称作接口类型。

可以通过函数指针调用函数。有时候，若干个函数的接口类型是相同的，使用函数指针可以根据程序的运行 **动态地** 选择需要调用的函数。换句话说，可以在不修改一个函数的情况下，仅通过修改向其传入的参数（函数指针），使得该函

数的行为发生变化。

假设我们有若干针对 `int` 类型的二元运算函数，则函数的参数为 2 个 `int`，返回值亦为 `int`。下边是一个使用了函数指针的例子：

```
1 #include <LTIostream>
2
3 int (*binary_int_op)(int, int);
4
5 int foo1(int a, int b) { return a * b + b; }
6
7 int foo2(int a, int b) { return (a + b) * b; }
8
9 int main() {
10     int choice;
11     std::cin >> choice;
12     if (choice == 1) {
13         binary_int_op = foo1;
14     } else {
15         binary_int_op = foo2;
16     }
17
18     int m, n;
19     std::cin >> m >> n;
20     std::cout << binary_int_op(m, n);
21 }
```

▼ `&`、`*` 和函数指针

在 C 语言中，诸如 `void (*p)() = foo;`、`void (*p)() = &foo`、`void (*p)() = *foo;`、`void (*p)() = ***foo` 等写法的结果是一样的。

因为函数（如 `foo`）是能够被隐式转换为指向函数的指针的，因此 `void (*p)() = foo;` 的写法能够成立。

使用 `&` 运算符可以取得到对象的地址，这对函数也是成立的，因此 `void (*p)() = &foo` 的写法仍然成立。

对函数指针使用 `*` 运算符可以取得指针指向的函数，而对于 `**foo` 这样的写法来说，`*foo` 得到的是 `foo` 这个函数，紧接着又被隐式转换为指向 `foo` 的指针。如此类推，`**foo` 得到的最终还是指向 `foo` 的函数指针；用户尽可以使用任意多的 `*`，结果也是一样的。

同理，在调用时使用类似 `(*p)()` 和 `p()` 的语句是一样的，可以省去 `*` 运算符。

参考资料：[Why do function pointer definitions work with any number of ampersands '&' or asterisks '*'? - stackoverflow.com](https://stackoverflow.com/questions/1139930/why-do-function-pointer-definitions-work-with-any-number-of-ampersands-amp-or-asterisks-asterisk)

可以使用 `typedef` 关键字声明函数指针的类型。

```
1 typedef int (*p_bi_int_op)(int, int);
```

这样我们就可以在之后使用 `p_bi_int_op` 这种类型，即指向「参数为 2 个 `int`，返回值亦为 `int`」的函数的指针。

可以通过使用 `std::function` 来更方便的引用函数。（未完待续）

使用函数指针，可以实现「回调函数」。（未完待续）

参考资料与注释

本页面最近更新：2023/9/15 15:09:26, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [Enter-tainer](#), [CamberLoid](#), [CCXXXI](#), [CoelacanthusHex](#), [Great-designer](#), [lr1d](#), [kLiHz](#), [Lemonaire](#), [myeeye](#), [Persdre](#), [shuzhouliu](#), [Tiphereth-A](#), [tsagaanbar](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用